

DI MIGRATION

Step-by-Step Implementation Guide

Grounded in your actual project files — every step tells you exactly which file, what to find, and what to write.

HOW TO USE THIS GUIDE

Each step shows: the exact file path → what to find → what to change or add. Steps are ordered so each one compiles and runs cleanly before you move to the next. Do not skip steps.

Overview — What You're Doing and Why

Your project already has the right structure: `ServiceLocator`, `ServiceDescriptor`, `ServiceScope`, `IInstaller`, `InstallerDependsOnAttribute`. None of that changes. The problems are:

#	Problem Today	What Breaks	Fixed In Step
1	Installers call <code>locator.Register(new Foo(...))</code> — you manually wire deps	If a dep isn't registered yet, you get null at runtime with no error	Steps 5–9
2	<code>InstallerDependsOnAttribute</code> is declared but never read	Installer order is whatever order Bootstrapper happens to run them in	Step 3
3	No boot-time validation — missing dep discovered mid-gameplay as <code>NullRef</code>	Hours debugging a <code>NullReferenceException</code> that should have been a startup error	Step 4
4	<code>CommandInstaller</code> casts locator to <code>(IServiceScope)</code> — breaks test doubles	Any mock locator that doesn't also implement <code>IServiceScope</code> crashes	Step 5
5	<code>TrackInitializable</code> calls <code>.Forget()</code> for late-resolved services	Async init failures are silently swallowed	Step 2
6	<code>ServiceScope.Dispose()</code> exists but never calls <code>IDisposable</code> on instances	Audio handles, file handles leak across scene loads	Step 1

Step

1

Fix ServiceScope — Make Dispose() Actually Dispose Instances

File: `Systems/Services/ServiceScope.cs` | ~10 lines changed

Right now `ServiceScope` implements `IDisposable` but the `Dispose()` method doesn't call `IDisposable` on any of the instances it holds. Audio services, file handles, and anything else that implements `IDisposable` leaks every time a scope ends.

 **FILE:** `Systems/Services/ServiceScope.cs`

 **FIND:** `private bool disposed = false;`

 **ACTION:** Replace the entire `Dispose()` method (or add it if missing) with this:

```
public void Dispose()
{
    if (disposed) return;
    disposed = true;

    // Dispose all scoped instances that implement IDisposable
    foreach (var instance in scopedInstances.Values)
    {
        if (instance is IDisposable disposable)
        {
            try { disposable.Dispose(); }
            catch (Exception ex)
            {
                Debug.LogError($"[ServiceScope] Dispose failed for" +
                    $" {instance.GetType().Name}: {ex.Message}");
            }
        }
    }

    scopedInstances.Clear();
    _scopedDescriptors.Clear();
}
```

 **VERIFY:** Project compiles. No behaviour change yet — this only runs when a scope is explicitly disposed, which nothing calls yet. You will wire that in Step 9.

Step

2

Fix TrackInitializable — Stop Swallowing Async Errors

File: `Systems/Services/ServiceLocator.cs` | ~15 lines changed

When a service is resolved after boot (e.g. from a scene installer), `TrackInitializable` calls `init.InitializeAsync().Forget()`. If that async method throws, the exception is silently swallowed — you get no error, no log, nothing.

 **FILE:** `Systems/Services/ServiceLocator.cs`

 **FIND:** `internal void TrackInitializable(object instance)`

 **ACTION:** Find the `if (isInitialized)` block inside `TrackInitializable`. Replace just that block:

```
internal void TrackInitializable(object instance)
{
    if (instance is not IInitializableService init) return;
    initializables.TryAdd(init, 0);

    if (isInitialized)
    {
        // BEFORE: init.InitializeAsync().Forget(); ← silent failure
        // AFTER: log the error so it's visible
        init.InitializeAsync()
            .ContinueWith(t =>
```

```

        {
            if (t.IsFaulted)
                Debug.LogError($"[ServiceLocator] Late-init failed for" +
                    $" {init.GetType().Name}: {t.Exception}");
        });
    }
}

```

NOTE

This uses a plain `.ContinueWith` instead of `.Forget()` so exceptions surface. If you want late-init failures to be fatal, change the `Debug.LogError` to `throw` — but logging is safer for now until all services are stable.

☑ **VERIFY:** Project compiles. Play mode enters. Late-resolved services that fail init will now print an error to the Console instead of silently failing.

Step

3

Create InstallerSorter — Enforce InstallerDependsOnAttribute

New file: Systems/Services/InstallerSorter.cs

`InstallerDependsOnAttribute` already exists at `Installers/InstallerDependsOnAttribute.cs`. `CommandInstaller` and `GameServicesInstaller` both use it. But nobody reads it — the attribute is decoration only. `InstallerSorter` reads the attributes and sorts the list so dependencies always run before dependents.

📄 **NEW FILE:** `Systems/Services/InstallerSorter.cs`

✏ **ACTION:** Create this file with exactly this content:

```

using System;
using System.Collections.Generic;
using System.Linq;
using Core.Installers;
using Core.Systems.Services.Interfaces;

namespace Core.Systems.Services
{
    public static class InstallerSorter
    {
        /// <summary>
        /// Sorts installers so that each installer runs after its
        /// [InstallerDependsOn] dependencies. Throws on circular or
        /// missing dependencies.
        /// </summary>
        public static List<IInstaller> Sort(IEnumerable<IInstaller> installers)
        {
            var all = installers.ToList();
            var byType = all.ToDictionary(i => i.GetType());

            // Build in-degree and edge map
            var inDegree = byType.Keys.ToDictionary(t => t, _ => 0);
            var edges = new Dictionary<Type, List<Type>>();

            foreach (var type in byType.Keys)
            {
                var attrs = type
                    .GetCustomAttributes(typeof(InstallerDependsOnAttribute), true)
                    .Cast<InstallerDependsOnAttribute>();
            }
        }
    }
}

```

```

        foreach (var attr in attrs)
        {
            var dep = attr.RequiredInstaller;
            if (!byType.ContainsKey(dep))
                throw new InvalidOperationException(
                    $"[InstallerSorter] {type.Name} depends on" +
                    $" {dep.Name} which is not in the installer list.");

            if (!edges.ContainsKey(dep))
                edges[dep] = new List<Type>();
            edges[dep].Add(type);
            inDegree[type]++;
        }
    }

    // Kahn's algorithm
    var queue = new Queue<Type>(
        inDegree.Where(kv => kv.Value == 0).Select(kv => kv.Key));
    var sorted = new List<IIInstaller>();

    while (queue.Count > 0)
    {
        var current = queue.Dequeue();
        sorted.Add(byType[current]);

        if (!edges.TryGetValue(current, out var dependents)) continue;
        foreach (var dep in dependents)
            if (--inDegree[dep] == 0)
                queue.Enqueue(dep);
    }

    if (sorted.Count != byType.Count)
        throw new InvalidOperationException(
            "[InstallerSorter] Circular dependency detected between installers.");

    return sorted;
}
}
}

```

☑ **VERIFY:** Project compiles. Nothing calls this yet — that happens in Step 10.

Step

4

Create DIValidator — Catch Missing Deps at Boot

New file: Systems/Services/DIValidator.cs

This is the key piece that eliminates `NullReferenceException` in gameplay from missing registrations. After all installers run, `DIValidator` walks every registered type, reads its constructor, and confirms every parameter type is also registered. If anything is missing, it throws with a clear message before the game starts.

It also catches the `[Inject]` readonly field bug in `UIAnimationService` (a readonly field that reflection cannot set — currently silently fails).

First, add `ImplementationType` to `ServiceDescriptor` — the validator needs to know the concrete class, not just the factory lambda.

 **FILE:** `Systems/Services/ServiceDescriptor.cs`

 **FIND:** `public Type ServiceType { get; }`

 **ACTION:** Add one new property on the next line:

```
public Type ServiceType { get; }
public Type ImplementationType { get; } // NEW – null for pre-built instances
```

 **FIND:** `public ServiceDescriptor(Type serviceType, Func<IServiceScope, object> factory, ServiceLifetime lifetime)`

 **ACTION:** Add a second constructor overload below the existing one:

```
// Existing constructor – keep it exactly as-is
public ServiceDescriptor(Type serviceType, Func<IServiceScope, object> factory,
    ServiceLifetime lifetime)
{
    ServiceType = serviceType;
    Factory = factory;
    Lifetime = lifetime;
}

// NEW – used by Register<TService, TImpl> to record the concrete type
public ServiceDescriptor(Type serviceType, Type implementationType,
    Func<IServiceScope, object> factory, ServiceLifetime lifetime)
    : this(serviceType, factory, lifetime)
{
    ImplementationType = implementationType;
}
```

 **FILE:** `Systems/Services/ServiceLocator.cs`

 **FIND:** `public void Register<TService, TImpl>(ServiceLifetime lifetime = ServiceLifetime.Singleton)`

 **ACTION:** Find the line inside this method that creates the `ServiceDescriptor` (it calls `Register<TService>(scope => ..., lifetime)`). Change it to use the new two-constructor overload so `ImplementationType` gets stored. Replace the whole `Register<TService, TImpl>` method body:

```
public void Register<TService, TImpl>(ServiceLifetime lifetime = ServiceLifetime.Singleton)
    where TImpl : class, TService
    where TService : class
{
    var implType = typeof(TImpl);
    var constructors = implType.GetConstructors();

    Func<IServiceScope, TService> factory;
    if (constructors.Length == 0)
    {
        factory = _ => Activator.CreateInstance<TImpl>();
    }
    else
    {
        var ctor = constructors.OrderByDescending(c => c.GetParameters().Length).First();
        var paramTypes = ctor.GetParameters().Select(p => p.ParameterType).ToArray();
        factory = scope =>
        {
            var args = paramTypes.Select(t => scope.Get(t)).ToArray();
            return (TService)ctor.Invoke(args);
        };
    }

    // Store ImplementationType so DIValidator can inspect it
}
```

```

var desc = new ServiceDescriptor(
    typeof(TService), implType, s => factory(s), lifetime);
descriptors[typeof(TService)] = desc;
Debug.Log($"[ServiceLocator] Registered {typeof(TImpl).Name} as" +
    $" {typeof(TService).Name} ({lifetime})");
}

```

NEW FILE: Systems/Services/DIValidator.cs

 **ACTION:** Create this file:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Reflection;
using Core.Utility.Attributes;
using UnityEngine;

namespace Core.Systems.Services
{
    public static class DIValidator
    {
        /// <summary>
        /// Call after all installers have run, before InitializeGlobalServicesWithProgress.
        /// Throws if any constructor dependency is unregistered.
        /// </summary>
        public static void ValidateRegistrations(ServiceLocator locator)
        {
            var errors = new List<string>();
            var warnings = new List<string>();

            foreach (var (_, desc) in locator.descriptors)
            {
                var implType = desc.ImplementationType;
                if (implType == null) continue; // pre-built instance, skip

                // --- Check constructor parameters ---
                var ctor = implType.GetConstructors()
                    .OrderByDescending(c => c.GetParameters().Length)
                    .FirstOrDefault();
                if (ctor == null) continue;

                foreach (var param in ctor.GetParameters())
                {
                    if (param.IsOptional) continue;
                    if (!locator.IsRegistered(param.ParameterType))
                        errors.Add(
                            $" {implType.Name} needs {param.ParameterType.Name}" +
                            $" ({param.Name}) – not registered.");
                }

                // --- Check [Inject] fields ---
                var fields = implType
                    .GetFields(BindingFlags.Instance | BindingFlags.NonPublic
                        | BindingFlags.Public)
                    .Where(f => f.GetCustomAttribute<InjectAttribute>() != null);

                foreach (var field in fields)
                {
                    if (field.IsInitOnly) // readonly
                        errors.Add(
                            $" {implType.Name}.{field.Name} is [Inject] readonly" +
                            " – reflection cannot set readonly fields." +

```

```

        " Use constructor injection instead.");
    else if (!locator.IsRegistered(field.FieldType))
        warnings.Add(
            $" {implType.Name}.{field.Name}" +
            $" ({field.FieldType.Name}) not registered.");
    }
}

foreach (var w in warnings)
    Debug.LogWarning($"[DValidator] {w}");

if (errors.Count == 0)
{
    Debug.Log($"[DValidator] All {locator.descriptors.Count}" +
        " registrations valid.");
    return;
}

var msg = $"[DValidator] {errors.Count} error(s) at boot:\n" +
    string.Join("\n", errors);
Debug.LogError(msg);
throw new InvalidOperationException(msg);
}
}
}

```

NAMESPACE NOTE The [Inject] attribute is at Core.Utility.Attributes.InjectAttribute — check the using path matches your project. If InjectAttribute lives somewhere else, adjust the using statement at the top.

✅ **VERIFY:** Project compiles. DValidator exists but nothing calls it yet. That happens in Step 10.

Step

5

Fix CommandInstaller — Remove the (IServiceScope) Cast

File: Installers/CommandInstaller.cs | 1 line changed

CommandInstaller casts locator to (IServiceScope) when constructing DebugConsoleService. ServiceLocator already implements IServiceScope, so this works today — but it breaks immediately if you ever use a test double or wrapper that only implements IServiceLocator.

 **FILE:** Installers/CommandInstaller.cs

 **FIND:** locator.Register(new DebugConsoleService((IServiceScope)locator));

 **ACTION:** Replace that one line with:

```
locator.Register(new DebugConsoleService(locator));
```

For this to compile, DebugConsoleService's constructor must accept IServiceLocator instead of IServiceScope. Open that file:

 **FILE:** Systems/DebugConsole/DebugConsoleService.cs

 **FIND:** public DebugConsoleService(IServiceScope

 **ACTION:** Change the constructor parameter type from IServiceScope to IServiceLocator:

```
// BEFORE
public DebugConsoleService(IServiceScope scope) { ... }

// AFTER
public DebugConsoleService(IServiceLocator locator) { ... }
```

Update the body to use the new parameter name. The call sites inside the class that use `scope.Get<T>()` still work because `IServiceLocator` exposes `Get<T>()`.

☑ **VERIFY:** Project compiles and play mode works. The cast is gone.

Step 6

Fix CoreServicesInstaller — Move Resources.Load() Out of Install()

File: *Installers/CoreServicesInstaller.cs* | ~20 lines moved

`CoreServicesInstaller.Install()` currently calls `Resources.Load<CoreConfiguration>()` at the top of the method. This is an untestable I/O side-effect inside registration. The `CoreConfiguration` should be loaded by the Bootstrapper and passed in as a constructor argument, just like `AudioServiceConfig` already is.

 **FILE:** `Installers/CoreServicesInstaller.cs`

 **FIND:** `private readonly IEnvironmentService _environmentService;`

 **ACTION:** Add a third field and update the constructor to accept `CoreConfiguration`:

```
private readonly IEnvironmentService _environmentService;
private readonly AudioServiceConfig _audioConfig;
private readonly CoreConfiguration _coreConfig; // NEW

public CoreServicesInstaller(
    IEnvironmentService environmentService,
    AudioServiceConfig audioConfig,
    CoreConfiguration coreConfig) // NEW PARAMETER
{
    _environmentService = environmentService;
    _audioConfig = audioConfig;
    _coreConfig = coreConfig; // NEW
}
```

 **FIND:** `var coreConfig = Resources.Load<CoreConfiguration>`

 **ACTION:** Delete the entire `Resources.Load` block at the top of `Install()` (the part that loads `CoreConfiguration` and falls back to `CreateInstance`). Then replace every use of the local variable `coreConfig` inside `Install()` with `_coreConfig`:

```
// DELETE these lines at the top of Install():
// var coreConfig = Resources.Load<CoreConfiguration>("Configuration/CoreConfiguration");
// if (coreConfig == null) { ... ScriptableObject.CreateInstance ... }

// All remaining references to 'coreConfig' in this method become '_coreConfig'
// Example:
var loggingConfig = _coreConfig.LoggingConfig;
// ... etc
```

Now update whoever creates `CoreServicesInstaller`. Find your Bootstrapper (likely in `Systems/Loading/Bootstrap/Bootstrapper.cs`):

 **FILE:** `Systems/Loading/Bootstrap/Bootstrapper.cs`

 **FIND:** `new CoreServicesInstaller(`

 **ACTION:** Load `CoreConfiguration` in the `Bootstrapper` before creating the installer, then pass it in:

```
// Add this BEFORE the installer is created:
var coreConfig = Resources.Load<CoreConfiguration>("Configuration/CoreConfiguration");
if (coreConfig == null)
{
    Debug.LogError("[Bootstrapper] CoreConfiguration not found at" +
        " Resources/Configuration/CoreConfiguration!");
    coreConfig = ScriptableObject.CreateInstance<CoreConfiguration>();
}

// Then pass it to the installer:
new CoreServicesInstaller(environmentService, audioConfig, coreConfig)
```

 **VERIFY:** Project compiles and boot works exactly as before, but `CoreConfiguration` is now loaded once in the `Bootstrapper`, not inside registration.

Step

7

Migrate `CoreServicesInstaller` to `Register<TService, TImpl>()`

File: `Installers/CoreServicesInstaller.cs` | Replace manual new with container registration

Now that the container has `Register<TService, TImpl>()` that auto-resolves constructor params, and `DValidator` will catch any mistakes, you can stop manually wiring dependencies in `CoreServicesInstaller`. The rule is simple:

- If the service's constructor only takes types that are registered in the container → use `Register<TService, TImpl>()`
- If the service needs a `ScriptableObject` (like `CoreConfiguration`, `AudioServiceConfig`, `Theme`) → either register the `ScriptableObject` first, or use the lambda overload `Register<T>(scope => new T(...))`

 **FILE:** `Installers/CoreServicesInstaller.cs`

 **ACTION:** Replace the entire `Install()` method body with this. Read the comments — they explain every decision:

```
public void Install(IServiceLocator locator)
{
    // — Data dependencies (ScriptableObjects) —————
    // Register these as instances so the container can resolve them
    // when building services that need them.
    locator.Register<CoreConfiguration>(_coreConfig);

    var loggingConfig = _coreConfig.LoggingConfig
        ?? ScriptableObject.CreateInstance<LoggingConfiguration>();
    locator.Register<LoggingConfiguration>(loggingConfig);

    // — Pure C# services – container resolves all ctor params —————
    locator.Register<ILoggerService, UnityLoggerService>();
    locator.Register<ITelemetryService, DebugTelemetryService>();
    locator.Register<IConfigurationService, ConfigurationService>();
    locator.Register<IEventBus, EventBus>();
    locator.Register<TaskRunner, TaskRunner>();
    locator.Register<AssetManagementService, AssetManagementService>();
}
```

```

locator.Register<IAsetManagementService>(
    scope => scope.Get<AssetManagementService>()); // alias – no double-init

// — Services needing ScriptableObject data – use lambda —————
locator.Register<Core.Systems.Theming.IThemeService>(
    scope => new Core.Systems.Theming.ThemeService(
        _coreConfig.Theme,
        scope.Get<ILoggerService>()));

locator.Register<Core.Systems.Juice.UIAnimation.UIAnimationService,
    Core.Systems.Juice.UIAnimation.UIAnimationService>();

locator.Register<PoolService, PoolService>();

// Apply pool definitions after PoolService is registered
// (this runs at registration time, before Init – safe because
// PoolService has no async init requirement for definitions)
locator.Register<Core.Systems.Settings.SettingsService,
    Core.Systems.Settings.SettingsService>();

// AudioService needs AudioServiceConfig (not a registered type) → lambda
locator.Register<AudioService>(scope =>
{
    AudioServiceConfig finalCfg = _audioConfig;
    if (_coreConfig.AudioConfig != null)
        finalCfg = AudioServiceConfig.FromScriptableObject(_coreConfig.AudioConfig);
    if (finalCfg == null) finalCfg = new AudioServiceConfig();

    return new AudioService(
        scope.Get<PoolService>(),
        scope.Get<Core.Systems.Settings.SettingsService>(),
        finalCfg,
        scope.Get<ILoggerService>());
});

// Apply pool definitions (was done inline before, still done here)
if (_coreConfig.PoolDefinitions != null)
{
    // PoolService is registered but not yet instantiated – store for post-init
    // The simplest approach: keep this logic but get the service lazily
    var poolService = locator.Get<PoolService>();
    foreach (var def in _coreConfig.PoolDefinitions)
        def.ApplyToPoolService(poolService);
}
}

```

WHY THIS COMPILES

Register<ILoggerService, UnityLoggerService>() works because UnityLoggerService's constructor only takes LoggingConfiguration, which is now registered as an instance above it. The container reads the constructor via reflection and calls locator.Get<LoggingConfiguration>() automatically.

☑ **VERIFY:** Boot works. Run the game and check the console — every [ServiceLocator] Registered line should appear in the same order as before. If anything throws 'not registered', check the order: a service registered via Register<T, TImpl> cannot have its deps registered after it.

Step

8

Migrate NavigationInstaller and GameServicesInstaller

Files: Installers/NavigationInstaller.cs, Installers/GameServicesInstaller.cs

NavigationInstaller

NavigationInstaller has two kinds of registrations: the TransitionRunner (a MonoBehaviour — must stay as an instance registration) and the pure C# services (NavigationService, PopupService — can be migrated).

 **FILE:** Installers/NavigationInstaller.cs

 **FIND:** // Navigation Service

 **ACTION:** Keep the TransitionRunner block exactly as-is (FindFirstObjectByType, Resources.Load, etc). Replace only the NavigationService and PopupService registrations:

```
// TransitionRunner – keep existing code unchanged (MonoBehaviour, must be instance)
// ...

// NavigationService – all deps are in the container
// Note: NavigationService constructor takes IServiceScope – ServiceLocator
// implements IServiceScope, so this cast is safe.
locator.Register<NavigationService>(scope =>
    new NavigationService(
        (Core.Systems.Services.Interfaces.IServiceScope)locator,
        scope.Get<TaskRunner>(),
        scope.Get<AssetManagementService>(),
        scope.Get<ITransitionService>(),
        scope.Get<Core.Systems.Logging.ILoggerService>()));

locator.Register<INavigationService>(
    scope => scope.Get<NavigationService>()); // alias

// PopupService – all deps in container
locator.Register<PopupService>(
    scope => new PopupService(
        scope,
        scope.Get<NavigationService>(),
        scope.Get<PoolService>()));
```

GameServicesInstaller

Most services here can be migrated. The SaveService stays conditional (it's already guarded by an if-block based on config). LanguageSettingsDefinition needs to be registered as a data dependency first since LocalizationService's constructor requires it.

 **FILE:** Installers/GameServicesInstaller.cs

 **ACTION:** Replace the Install() method body:

```
public void Install(IServiceLocator locator)
{
    // — Data dependencies —————
    var languageSettings = _coreConfig.LanguageSettings
        ?? Resources.Load<LanguageSettingsDefinition>(
            "Localization/LanguageSettings");
    locator.Register<LanguageSettingsDefinition>(languageSettings);

    SaveSystemConfiguration finalSaveConfig = _saveConfig;
    if (_coreConfig.SaveSystemConfig != null)
        finalSaveConfig = _coreConfig.SaveSystemConfig;
    locator.Register<SaveSystemConfiguration>(finalSaveConfig);

    // — Pure C# services —————
    locator.Register<IWebRequestService, WebRequestService>();
```

```

locator.Register<LocalizationService, LocalizationService>();
locator.Register<IErrorHandler, ErrorHandler>();
locator.Register<RebindService, RebindService>();

// LoadingService takes IServiceScope – use lambda
locator.Register<LoadingService>(
    scope => new LoadingService(
        scope.Get<TaskRunner>(),
        scope.Get<NavigationService>(),
        (IServiceScope)locator,
        scope.Get<IConfigurationService>(),
        scope.Get<Core.Systems.Logging.ILoggerService>()));

// — Save System —————
var envService = locator.Get<Core.Systems.Environment.IEnvironmentService>();
var profile = finalSaveConfig.GetProfileForEnvironment(envService.Current);
if (profile.serviceType == SaveServiceType.Generic)
{
    locator.Register<ISaveService>(
        scope => SaveSystemFactory.Create(
            scope.Get<SaveSystemConfiguration>(),
            scope.Get<Core.Systems.Environment.IEnvironmentService>(),
            scope.Get<Core.Systems.Logging.ILoggerService>()));
}
}

```

IMPORTANT WebRequestService, LocalizationService, ErrorHandler, and RebindService must have constructors that only take types now registered. If any of these have extra ScriptableObject or config params, you will see a DIValidator error at boot telling you exactly what is missing — which is the whole point.

☑ **VERIFY:** Boot works. All services resolve. Check console for [DIValidator] All registrations valid. (you will add that call in Step 10).

Step

9

Fix UIAnimationService — Remove [Inject] readonly Field

File: Systems/Juice/UIAnimation/UIAnimationService.cs | ~5 lines changed

UIAnimationService has a [Inject] readonly field for TaskRunner. The [Inject] mechanism uses reflection to set the field after construction — but readonly fields cannot be set by reflection in modern .NET. The injection silently does nothing. TaskRunner is null. Any animation that uses TaskRunner crashes at runtime, not at boot.

Since you migrated CoreServicesInstaller to Register<UIAnimationService, UIAnimationService>() in Step 7, the container will now call the constructor — so TaskRunner just needs to be a constructor parameter.

 **FILE:** Systems/Juice/UIAnimation/UIAnimationService.cs

 **FIND:** [Inject] private readonly TaskRunner

 **ACTION:** Delete the [Inject] field. Find the constructor and add TaskRunner as a required parameter:

```

// DELETE this line:
// [Inject] private readonly TaskRunner taskRunner;

// Find the backing field (probably already exists as _taskRunner or similar)
// and make sure it's assigned in the constructor:

private readonly TaskRunner _taskRunner; // keep this

```

```
public UIAnimationService(TaskRunner taskRunner) // ensure this exists
{
    _taskRunner = taskRunner
        ?? throw new ArgumentNullException(nameof(taskRunner));
    // ... rest of constructor ...
}
```

☑ **VERIFY:** DIValidator (in Step 10) will confirm TaskRunner is properly wired. UIAnimationService._taskRunner will be non-null after construction.

Step 10

Wire Everything in the Bootstrapper

File: Systems/Loading/Bootstrap/Bootstrapper.cs | The final connection

This is the last step. The Bootstrapper needs to: (1) sort installers using InstallerSorter, (2) run them, (3) validate with DIValidator before anything initializes. Find the method in your Bootstrapper that creates and runs the installers — it likely looks something like a list of new XxxInstaller() calls followed by Install(locator).

 **FILE:** Systems/Loading/Bootstrap/Bootstrapper.cs

 **FIND:** new CoreServicesInstaller(

 **ACTION:** Replace the installer creation and execution block with this pattern. Adapt the installer list to match what your Bootstrapper already creates — just change how they're run:

```
// 1. Load CoreConfiguration (you added this in Step 6)
var coreConfig = Resources.Load<CoreConfiguration>("Configuration/CoreConfiguration");
if (coreConfig == null)
{
    Debug.LogError("[Bootstrapper] CoreConfiguration missing!");
    coreConfig = ScriptableObject.CreateInstance<CoreConfiguration>();
}

// 2. Build the installer list (same installers as before, just in a list)
var installers = new List<IInstaller>
{
    new CoreServicesInstaller(environmentService, audioConfig, coreConfig),
    new NavigationInstaller(),
    new GameServicesInstaller(coreConfig, saveConfig),
    new CommandInstaller(),
    // add any other installers here
};

// 3. Sort by [InstallerDependsOn] – this replaces your manual ordering
var sorted = InstallerSorter.Sort(installers);

// 4. Collect errors rather than crashing on first failure
var bootErrors = new List<string>();
foreach (var installer in sorted)
{
    try { installer.Install(locator); }
    catch (Exception ex)
    {
        bootErrors.Add($"{installer.GetType().Name}: {ex.Message}");
        Debug.LogError($"[Bootstrapper] {installer.GetType().Name} failed: {ex}");
    }
}
```

```
// 5. Validate – catches missing deps, [Inject] readonly fields, etc.
//    Runs even if some installers failed so you see ALL problems at once.
try { DIValidator.ValidateRegistrations(locator); }
catch (InvalidOperationException ex)
{
    bootErrors.Add(ex.Message);
}

// 6. If any errors, throw now – before the game starts
if (bootErrors.Count > 0)
    throw new Exception(
        $"[Bootstrapper] {bootErrors.Count} boot error(s):\n" +
        string.Join("\n", bootErrors));

// 7. Initialize all IInitializableService implementations
await locator.InitializeGlobalServicesWithProgress(OnProgress);

// Everything from here runs on a guaranteed-clean graph.
```

Add the required using statements at the top of Bootstrapper.cs if they aren't there:

```
using System.Collections.Generic;
using Core.Systems.Services; // InstallerSorter, DIValidator
```

WHAT HAPPENS IF SOMETHING IS MISSING If you forgot to register a type, you will now see: [DIValidator] 1 error(s) at boot: FooService needs BarService (barService) — not registered. This throws before the first scene loads. No more hunting NullReferenceException in gameplay.

☒ **VERIFY:** Boot. Watch the console. You should see: [DIValidator] All N registrations valid. then normal boot. Done.

Quick Reference — Registration Patterns

Now that the container owns wiring, here is when to use each registration style:

Situation	Use This	Example
All constructor params are registered services	<code>locator.Register<IFoo, FooImpl>();</code>	LoggerService, WebRequestService, EventBus
Service needs a ScriptableObject or config data	<code>locator.Register<IFoo>(scope => new FooImpl(myData, scope.Get<IBar>()));</code>	AudioService, ThemeService, LocalizationService
Pre-built instance (MonoBehaviour, pre-constructed)	<code>locator.Register<IFoo>(existingInstance);</code>	TransitionRunner, any MonoBehaviour
Alias — two keys, one instance	<code>locator.Register<IFoo>(s => s.Get<FooImpl>());</code>	IAssetManagementService → AssetManagementService

ScriptableObject that other services depend on	locator.Register<MyConfig>(myScriptableObject);	CoreConfiguration, LanguageSettingsDefinition
--	---	---

Troubleshooting — Errors You Will See and What They Mean

[DIValidator] FooService needs BarService (barService) – not registered.

BarService is not registered before FooService's Install() runs. Either add `locator.Register<BarService, BarService>()` before FooService, or add `[InstallerDependsOn(typeof(BarInstaller))]` to the installer that registers FooService.

[InstallerSorter] FooInstaller depends on BarInstaller which is not in the installer list.

FooInstaller has `[InstallerDependsOn(typeof(BarInstaller))]` but you didn't add BarInstaller to the installer list in Bootstrapper. Add it.

[DIValidator] UIAnimationService._taskRunner is [Inject] readonly – reflection cannot set readonly fields.

You still have the `[Inject]` readonly field. Go back to Step 9 and move TaskRunner to the constructor.

[ServiceLocator] Circular dependency detected: FooService -> BarService -> FooService

FooService and BarService depend on each other. One of them needs to take a factory/lazy wrapper, or the dependency is wrong and should be inverted.